

SpringMVC

鸣谢：黑马程序员



黑马程序员SSM框架教程_Spring+SpringMVC+Maven高级+SpringBo...

UP 黑马程序员 · 2022-4-20

SpringMVC

一、MVC

1.概述

2.组件职能

2.1 模型

2.2 视图

2.3 控制器

二、SpringMVC入门

1.概述

2.入门案例

2.1 使用SpringMVC技术需要先导入SpringMVC坐标与Servlet坐标

2.2 创建SpringMVC控制器类

2.3 初始化SpringMVC环境，让SpringMVC加载对应的 bean

2.4 初始化Servlet容器，加载SpringMVC环境，并设置SpringMVC要处理的请求

3.入门案例的工作流程

3.1 启动服务器并初始化

3.2 单次请求

4.同时加载Spring和SpringMVC

4.1 重构 `ServletContainerInitConfig` 类

4.2 `Controller` 与业务 `Bean` 的加载控制

4.3 简化4.1代码

三、请求与响应

1.请求映射路径： `@RequestMapping`

2.GET请求传参

2.1 普通参数

P1 概述

P2 请求参数名与形参名不一致的情况： `@RequestParam`

2.2 POJO参数

2.3 嵌套POJO参数

2.4 数组参数

2.5 集合参数： `@RequestParam`

2.6 通过json格式传参： `@RequestBody`

P0 准备工作

P1 集合参数

P2 POJO参数

P3 POJO集合参数

2.7 日期参数

3.POST请求传参

P1 概述

P2 POST请求中文乱码处理

2.8 类型转换器

4.响应json数据

4.0 准备工作

4.1 代码演示

4.2 @ResponseBody 注解

四、REST风格

1.简介

2.RESTful入门案例: @PathVariable

2.1 代码演示

2.2 注解区别

3.RESTful快速开发: @RestController、@RequestMapping

4.案例: 基于RESTful的页面数据交互

五、拦截器

1.概述

2.入门案例

2.1 创建拦截器类 ProjectInterceptor, 并实现 HandlerInterceptor 接口

2.2 创建配置类 SpringMvcSupport, 继承 WebMvcConfigurationSupport, 并实现 addInterceptors() 方法

2.3 在 SpringMvcConfig 中扫描 com.zsh.config 这个包使拦截器生效

2.4 使用标准接口 WebMvcConfigurer 简化2.2+2.3

2.5 拦截器执行流程

3.拦截器参数

3.1 前置处理器

3.2 后置处理器

3.3 完成后处理器

4.拦截器链的执行顺序

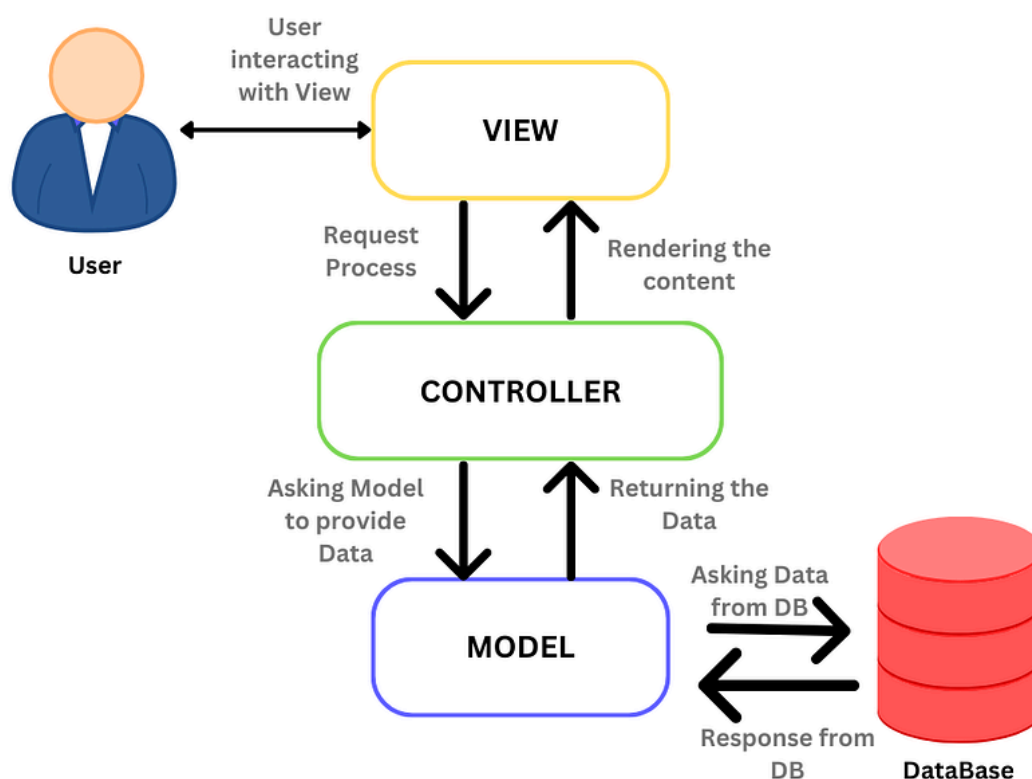
一、MVC

1.概述

- MVC(Model-View-Controller)是一种广泛使用的软件架构模式, 用于组织应用程序的代码结构, 使其更加模块化, 增强软件系统的可维护性和可扩展性。

- MVC将应用程序分为3个主要组件：Model（模型）、View（视图）和Controller（控制器）。

2.组件职能



图源: <https://medium.com/@sadikarahmantanisha/the-mvc-architecture-97d47e071eb2>

2.1 模型

- 模型是MVC架构中的**核心**。
- 主要负责数据和业务逻辑：
 - 与数据库或其他数据源交互，处理数据的CRUD操作。
- 模型与视图和控制器相互独立，不直接与用户界面相关联。
- 模型的变化通常由控制器触发，并最终反映到视图中。

2.2 视图

- 视图是MVC架构中的**表现层**。
- 主要负责将模型中的数据以用户可见的形式展示到用户界面：
 - 接收控制器传递的数据，进行展示和简单格式化。
- 视图通常是静态的，不包含业务逻辑。

2.3 控制器

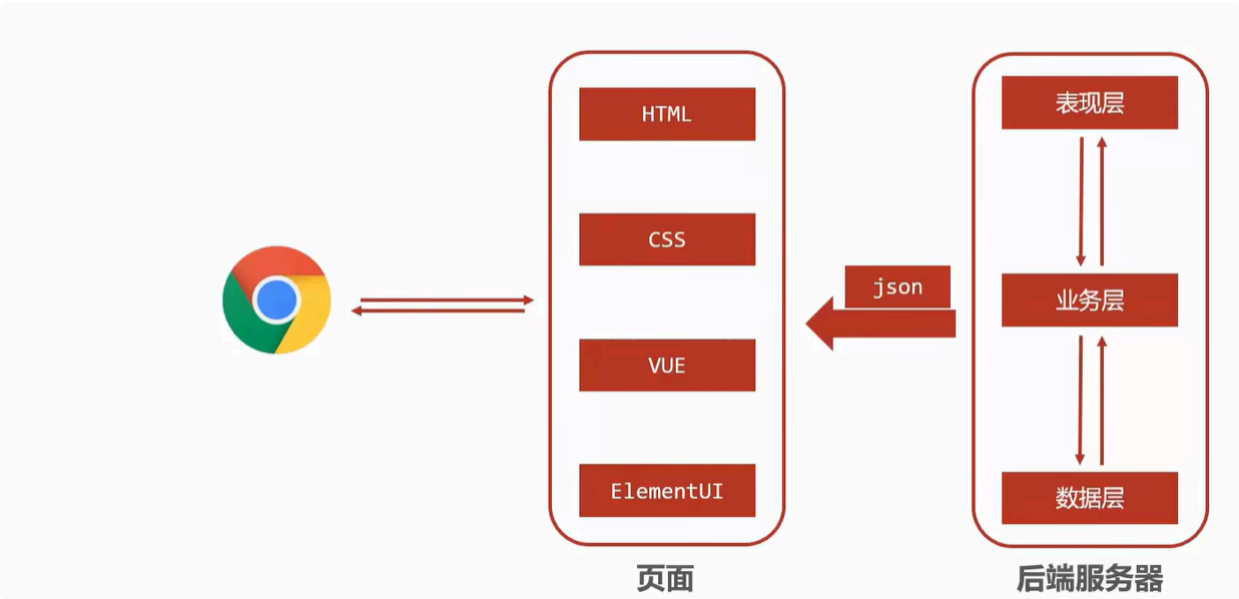
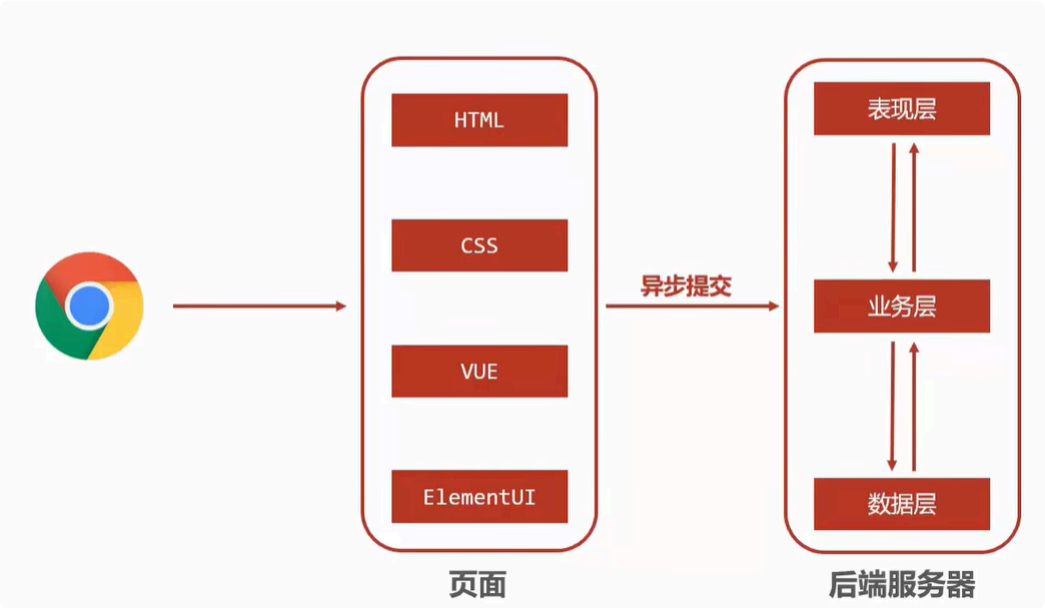
- 控制器是模型和视图的**中介**。
 - 主要负责处理用户请求：
 - 接收用户请求（如点击、表单提交），并调用相应的模型方法处理数据，根据处理结果选择合适的视图，并将数据传递给视图进行展示。
-

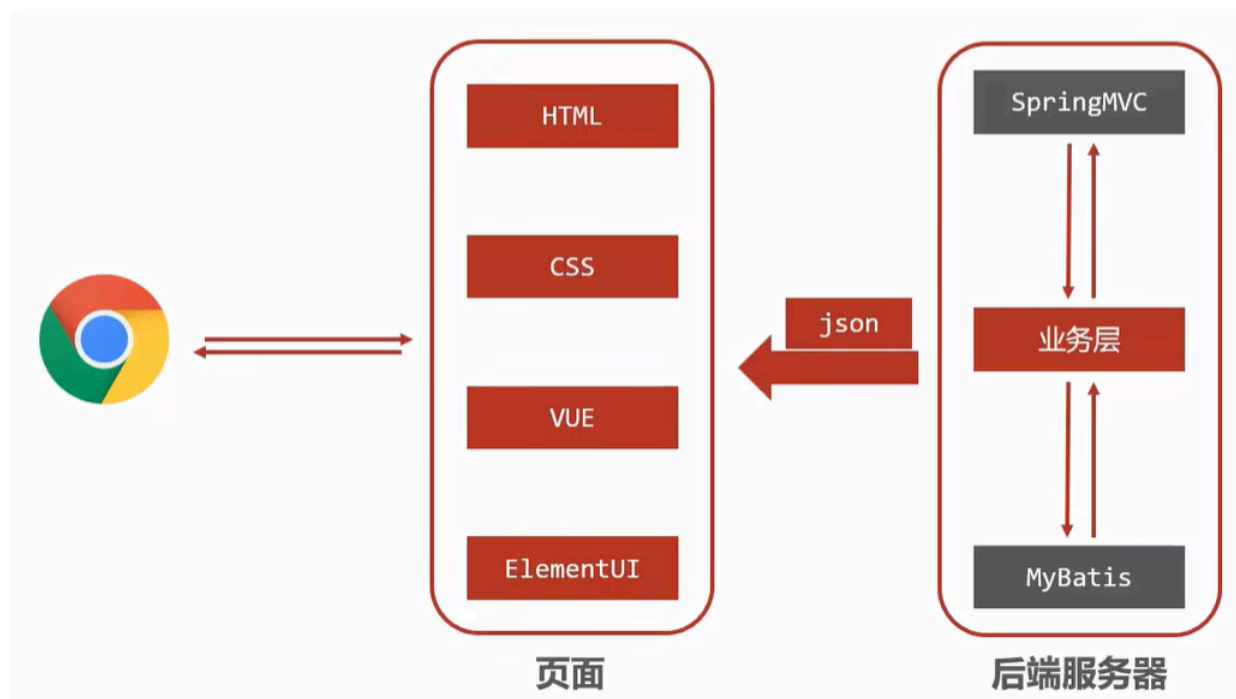
二、SpringMVC入门

💡 Tip

SpringMVC技术与Servlet技术等同，均属于Web层（别名：表现层、接口层）开发技术，前者明显优于后者。

1.概述





- SpringMVC是一种基于Java实现MVC软件架构模型的轻量级**Web框架**（别名：**表现层框架、接口层框架**）。
- 优点：使用简单，开发便捷，灵活性强。

2. 入门案例

2.1 使用SpringMVC技术需要先导入SpringMVC坐标与Servlet坐标

```
1  <!--1.导入SpringMVC与Servlet的坐标-->
2  <dependency>
3      <groupId>javax.servlet</groupId>
4      <artifactId>javax.servlet-api</artifactId>
5      <version>版本</version>
6      <scope>provided</scope>
7  </dependency>
8  <dependency>
9      <groupId>org.springframework</groupId>
10     <artifactId>spring-webmvc</artifactId>
11     <version>版本</version>
```

2.2 创建SpringMVC控制器类

```
1 // 2.定义controller
2 @Controller// 使用@Controller来定义bean
3 public class UserController {
4
5     @RequestMapping("/save")// 设置当前操作的访问路径
6     @ResponseBody// 将方法返回值设置为当前操作的响应体
7     public String save(){
8         System.out.println("user save ...");
9         return "{ 'module': 'springmvc' }";
10    }
11 }
```

2.3 初始化SpringMVC环境，让SpringMVC加载对应的 bean

```
1 // 3.创建SpringMVC的配置文件，加载Controller对应的bean
2 @Configuration
3 @ComponentScan("com.zsh.controller")
4 public class SpringMvcConfig {
5 }
```

2.4 初始化Servlet容器，加载SpringMVC环境，并设置SpringMVC要处理的请求

```
1 // 4.定义一个Servlet容器启动的配置类，在里面加载SpringMVC的配置
2 public class ServletContainerInitConfig extends
   AbstractDispatcherServletInitializer{
3     @Override
4     // 加载SpringMVC容器配置
5     protected WebApplicationContext createServletApplicationContext() {
6         AnnotationConfigWebApplicationContext context=new
   AnnotationConfigWebApplicationContext();// 注意类名不要写错，有个Web
7         context.register(SpringMvcConfig.class);
8         return context;
9     }
```



```
10
11     @Override
12     // 设置哪些请求交由SpringMVC处理
13     protected String[] getServletMappings() {
14         return new String[]{"/*"};
15     }
16
17     @Override
18     // 加载Spring容器配置
19     protected WebApplicationContext createRootApplicationContext() {
20         return null;
21     }
22 }
```

3. 入门案例的工作流程

3.1 启动服务器并初始化



1. 服务器启动，执行 `ServletContainerInitConfig` 类，初始化Web容器。
2. 执行 `ServletContainerInitConfig` 类中的 `createServletApplicationContext()` 方法，创建一个 `WebApplicationContext` 对象。
3. 加载 `SpringMvcConfig` 类。
4. 通过 `SpringMvcConfig` 类中的 `@ComponentScan` 注解加载对应的 `bean`。
5. 加载 `UserController` 类，每个 `@RequestMapping` 的名称对应一个具体方法。

6. 执行 `ServletContainerInitConfig` 类中的 `getServletMappings()` 方法，设置所有请求都交由SpringMVC处理。

3.2 单次请求

1. 发送请求 `localhost:8082/save`。（8082是我设置的 `Tomcat` 端口，可以自定义修改）
2. Web容器将所有请求都交由SpringMVC处理。
3. 解析请求路径 `/save`。
4. `/save` 映射到对应方法 `save()`。
5. 执行 `save()` 方法。
6. 检测到 `save()` 方法带有 `@ResponseBody` 注解，直接将 `save()` 方法的返回值作为响应体返回给请求方。

4.同时加载Spring和SpringMVC

4.1 重构 `ServletContainerInitConfig` 类

```
1 public class ServletContainerInitConfig extends
  AbstractDispatcherServletInitializer{
2     @Override
3     protected WebApplicationContext createRootApplicationContext() {
4         // Root
5         AnnotationConfigWebApplicationContext context=new
  AnnotationConfigWebApplicationContext();
6         context.register(SpringConfig.class); // 加载Spring
7         return context;
8     }
9     @Override
10    protected WebApplicationContext createServletApplicationContext()
11    {
12        // Servlet
13        AnnotationConfigWebApplicationContext context=new
  AnnotationConfigWebApplicationContext();
14        context.register(SpringMvcConfig.class); // 加载SpringMVC
15        return context;
16    }
17 }
```

```

14     }
15
16     @Override
17     protected String[] getServletMappings() {
18         return new String[]{"/*"};
19     }
20 }

```

4.2 Controller 与业务 Bean 的加载控制

- SpringMVC加载的 bean 均在 `com.zsh.controller` 包内。
- 如何避免Spring错误加载SpringMVC控制的 bean？
 - **方式一（推荐）**：Spring加载的 bean 设定扫描范围为精确范围。如 `@ComponentScan({"com.zsh.service", "com.zsh.dao"})`。
 - **方式二**：Spring加载的 bean 设定扫描范围为 `com.zsh`，同时排除掉 `controller` 包内的 bean。

```

1 // @Configuration这个注解必须去掉，否则过滤无效
2 @ComponentScan("com.zsh.controller")
3 public class SpringMvcConfig {
4 }

```

```

1 @Configuration
2 @Component(value = "com.zsh",
3             excludeFilters = @ComponentScan.Filter(
4                 type = FilterType.ANNOTATION, // 按注解过滤
5                 classes = Controller.class // 过滤掉带@Controller注解的bean
6             )
7 )
8 public class SpringConfig {
9 }

```

- **方式三**：不区分Spring与SpringMVC的环境，加载到同一环境中。

4.3 简化4.1代码

```
1 public class ServletContainerInitConfig extends
  AbstractAnnotationConfigDispatcherServletInitializer{
2
3     @Override
4     protected Class<?>[] getRootConfigClasses() {
5         return new Class[]{SpringConfig.class};
6     }
7
8     @Override
9     protected Class<?>[] getServletConfigClasses() {
10        return new Class[]{SpringMvcConfig.class};
11    }
12
13    @Override
14    protected String[] getServletMappings() {
15        return new String[]{"/"};
16    }
17 }
```

三、请求与响应

1.请求映射路径：@RequestMapping

@RequestMapping 注解既可用于方法上，也可用于整个类上，这时候的作用是统一设置当前类的方法的请求映射路径前缀。示例如下：

```
1 @Controller
2 @RequestMapping("/user")// 请求路径前缀
3 public class UserController {
4
5     @RequestMapping("/save")
6     @ResponseBody
7     public String save(){
8         System.out.println("user save ...");
9     }
10 }
```

```
9         return "{ 'module': 'user save' }";
10     }
11
12     @RequestMapping("/delete")
13     @ResponseBody
14     public String delete(){
15         System.out.println("user delete ...");
16         return "{ 'module': 'user delete' }";
17     }
18 }
```

2.GET请求传参

2.1 普通参数

P1 概述

传参方式：url地址直接传参，地址参数名与形参名一致，在方法签名里定义形参即可接收参数。

GET

▼

{{localhost}}/commonParam?name=zsh&age=20

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

☒	Key	Value
☒	name	zsh
☒	age	20
	Key	Value



```

1  @Controller
2  public class UserController {
3      // 普通参数
4      @RequestMapping("/commonParam")
5      @ResponseBody
6      public String commonParam(String name, int age){
7          System.out.println("common param: name ==> "+name);
8          System.out.println("common param: age ==> "+age);
9          return "{\"module':'common param'}";
10     }
11 }

```

P2 请求参数名与形参名不一致的情况: @RequestParam

```

1  @Controller
2  public class UserController {
3      // 普通参数: 请求参数名与形参名不一致的情况
4      @RequestMapping("/commonParamDifferentName")
5      @ResponseBody
6      public String commonParamDifferentName(@RequestParam("name") String
7      userName, int age){
8          System.out.println("common param: name ==> "+userName);
9          System.out.println("common param: age ==> "+age);
10         return "{\"module':'common param different name'}";
11     }
12 }

```

2.2 POJO参数

💡 Tip

POJO(Plain Ordinary Java Object): 简单的Java对象, 实际上就是普通的JavaBean, 只是为了避免和EJB混淆所创造的简称。

GET

▼

{{localhost}}/pojoParam?name=zsh&age=20

- ☰ Docs
- Params ●
- Authorization
- Headers (6)
- Body
- Scripts
- Settings

Query Params

☒	Key	Value
☒	name	zsh
☒	age	20
	Key	Value

```
1 | @Controller
2 | public class UserController {
3 |     // POJO参数
4 |     @RequestMapping("/pojoParam")
5 |     @ResponseBody
6 |     public String pojoParam(User user){
7 |         System.out.println("pojo param: user ==> "+user);
8 |         return '{"module':'pojo param'}';
9 |     }
10 | }
```

2.3 嵌套POJO参数

GET

▼

{{localhost}}/pojoContainsPojoParam?name=zsh&age=20&address.province=Guangdong&address.city=Swatow

☰ Docs

Params ●

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

☒	Key	Value	Descrip
☒	name	zsh	
☒	age	20	
☒	address.province	Guangdong	
☒	address.city	Swatow	
	Key	Value	Descrip

```
1 @Controller
2 public class UserController {
3     // 嵌套POJO参数
4     @RequestMapping("/pojoContainsPojoParam")
5     @ResponseBody
6     public String pojoContainsPojoParam(User user){
7         System.out.println("pojo param: user ==> "+user);
8         return "{\"module':'pojo contains pojo param'}";
9     }
10 }
```


2.4 数组参数

GET ▼ `{{localhost}}/arrayParam?hobbies=sing&hobbies=dance&hobbies=rap`

Docs Params ● Authorization Headers (6) Body Scripts Settings

Query Params

☒	Key	Value
☒	hobbies	sing
☒	hobbies	dance
☒	hobbies	rap
	Key	Value

```
1 @Controller
2 public class UserController {
3     // 数组参数
4     @RequestMapping("/arrayParam")
5     @ResponseBody
6     public String arrayParam(String[] hobbies){
7         System.out.println("array param: hobbies ==> "+
8 Arrays.toString(hobbies));
9         return "{ 'module': 'array param' }";
10    }
```

2.5 集合参数: @RequestParam

GET ⌵ `{{localhost}}/listParam?hobbies=sing&hobbies=dance&hobbies=rap&hobbies=basketball`

☰ Docs **Params** ● Authorization Headers (6) Body Scripts Settings

Query Params

☒	Key	Value
☒	hobbies	sing
☒	hobbies	dance
☒	hobbies	rap
☒	hobbies	basketball
	Key	Value

```
1 @Controller
2 public class UserController {
3     // 集合参数
4     @RequestMapping("/listParam")
5     @ResponseBody
6     public String listParam(@RequestParam List<String> hobbies){// 此处
// 必须加@RequestParam注解来绑定参数关系，否则报错
7         System.out.println("list param: hobbies ==> "+ hobbies);
8         return "{ 'module': 'list param' }";
9     }
10 }
```

2.6 通过json格式传参: @RequestBody

P0 准备工作

首先要导入json坐标，才能进行接下来的步骤！

```
1 <dependencies>
2   <dependency>
3     <groupId>com.fasterxml.jackson.core</groupId>
4     <artifactId>jackson-databind</artifactId>
5     <version>版本</version>
6   </dependency>
7 </dependencies>
```

其次要在 `SpringMvcConfig` 中新增注解 `@EnableWebMvc` !

```
1 @Configuration
2 @ComponentScan("com.zsh.controller")
3 @EnableWebMvc// 其中一个功能: 对json数据进行自动类型转换
4 public class SpringMvcConfig {
5 }
```

P1 集合参数

GET

▼

{{localhost}}/listParamForJson

☰ Docs

Params

Authorization

Headers (8)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ▼

1

["swim","run","fly"]

```

1  @Controller
2  public class UserController {
3      // 集合参数: json
4      @RequestMapping("/listParamForJson")
5      @ResponseBody
6      public String listParamForJson(@RequestBody List<String> hobbies)
7      { // 此处必须加@RequestBody注解来绑定参数关系
8          System.out.println("list param json: hobbies ==> " + hobbies);
9          return "{ 'module': 'list param for json' }";
10     }
11 }

```

P2 POJO参数

GET

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```

1  {
2      "name": "zsh",
3      "age": 99,
4      "address": {
5          "province": "Guangdong",
6          "city": "Swatow"
7      }
8  }

```

```

1  @Controller
2  public class UserController {
3      // POJO参数: json
4      @RequestMapping("/pojoParamForJson")
5      @ResponseBody
6      public String pojoParamForJson(@RequestBody User user) { // 此处必须加
7      @RequestBody注解来绑定参数关系
8          System.out.println("pojo param json: user ==> " + user);
9          return "{ 'module': 'pojo param for json' }";
10     }
11 }

```

P3 POJO集合参数

GET

▼

{{localhost}}/pojoListParamForJson

☰ Docs

Params

Authorization

Headers (8)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ▼

1

[

2

{"name": "zsh", "age": 66},

3

{"name": "zjl", "age": 100}

4

]

```
1 @Controller
2 public class UserController {
3     // POJO集合参数: json
4     @RequestMapping("/pojoListParamForJson")
5     @ResponseBody
6     public String pojoListParamForJson(@RequestBody List<User> users)
7     { // 此处必须加@RequestBody注解来绑定参数关系
8         System.out.println("pojo list param json: users ==> " + users);
9         return "{ 'module': 'pojo list param for json' }";
10    }
11 }
```

2.7 日期参数

💡 Tip

Spring默认的标准日期格式是 yyyy/MM/dd。

GET

`{{localhost}}/dateParam?date1=2088/03/17&date2=2088-03-17&date3=2088-03-17 09:05:20`

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

Query Params

☒	Key	Value
☒	date1	2088/03/17
☒	date2	2088-03-17
☒	date3	2088-03-17 09:05:20
	Key	Value

```
1  @Controller
2  public class UserController {
3      // 日期参数
4      @RequestMapping("/dateParam")
5      @ResponseBody
6      public String dateParam(
7          Date date1,
8          @DateTimeFormat(pattern = "yyyy-MM-dd") Date date2,
9          @DateTimeFormat(pattern = "yyyy-MM-dd HH:mm:ss") Date date3
10     ){
11         System.out.println("date param: date1 ==> "+date1);
12         System.out.println("date param: date2(yyyy-MM-dd) ==> "+date2);
13         System.out.println("date param: date3(yyyy-MM-dd HH:mm:ss) ==>
14         "+date3);
14         return "{ 'module': 'date param' }";
15     }
16 }
```

3.POST请求传参

P1 概述

传参方式：请求体里通过 `x-www-form-urlencoded` 表单传参，表单参数名与形参名一致，在方法签名里定义形参即可接收参数。

POST

▼

{{localhost}} /commonParam

Docs

Params

Authorization

Headers (8)

Body ●

Scripts

Settings

☐ none

☐ form-data

☒ x-www-form-urlencoded

☐ raw

☐ binary

☐ GraphQL

	Key	Value
☒	name	zsh
☒	age	30
	Key	Value

```
1 @Controller
2 public class UserController {
3
4     // 普通参数
5     @RequestMapping("/commonParam")
6     @ResponseBody
7     public String commonParam(String name, int age){
8         System.out.println("common param: name ==> "+name);
9         System.out.println("common param: age ==> "+age);
10        return "{\"module':'common param'}";
11    }
12 }
```

P2 POST请求中文乱码处理

```
1 public class ServletContainerInitConfig extends
  AbstractAnnotationConfigDispatcherServletInitializer {
2
3     @Override
4     protected Class<?>[] getServletConfigClasses() {
5         return new Class[]{SpringMvcConfig.class};
6     }
7
8     @Override
9     protected String[] getServletMappings() {
10         return new String[]{"/"};
11     }
12
13     @Override
14     protected Class<?>[] getRootConfigClasses() {
15         return new Class[0];
16     }
17
18     // POST请求中文乱码处理：设置过滤器
19     @Override
20     protected Filter[] getServletFilters() {
21         CharacterEncodingFilter filter=new CharacterEncodingFilter();
22         filter.setEncoding("UTF-8");
23         return new Filter[]{filter};
24     }
25 }
```


2.8 类型转换器

类型转换器

- Converter接口

```
public interface Converter<S, T> {  
    @Nullable  
    T convert(S var1);  
}
```

- 请求参数年龄数据 (String→Integer)
- 日期格式转换 (String → Date)

- @EnableWebMvc功能之一：根据类型匹配对应的类型转换器

4.响应json数据

4.0 准备工作

首先要导入json坐标，才能进行接下来的步骤！

```
1 <dependencies>  
2     <dependency>  
3         <groupId>com.fasterxml.jackson.core</groupId>  
4         <artifactId>jackson-databind</artifactId>  
5         <version>版本</version>  
6     </dependency>  
7 </dependencies>
```

其次要在 `SpringMvcConfig` 中新增注解 `@EnableWebMvc` ！

```
1 @Configuration
2 @ComponentScan("com.zsh.controller")
3 @EnableWebMvc// 其中一个功能：对json数据进行自动类型转换
4 public class SpringMvcConfig {
5 }
```

4.1 代码演示

- `page.jsp`:

```
1 <html>
2 <body>
3 <h2>Hello SpringMVC!</h2>
4 </body>
5 </html>
```

- `UserController`:

```
1 @Controller
2 public class UserController {
3     // 响应页面/跳转页面
4     @RequestMapping("/toJumpPage")
5     public String toJumpPage(){
6         System.out.println("jump page");
7         return "page.jsp";
8     }
9
10    // 响应文本数据
11    @RequestMapping("/toText")
12    @ResponseBody
13    public String toText(){
14        System.out.println("return text");
15        return "response text";
16    }
17
18    // 响应POJO: json
19    @RequestMapping("/toJsonPojo")
20    @ResponseBody
21    public User toJsonPojo(){
22        System.out.println("return json pojo");
```

```

23         User user=new User("zsh",999);
24         return user;
25     }
26
27     // 响应POJO集合: json
28     @RequestMapping("/toJsonPojoList")
29     @ResponseBody
30     public List<User> toJsonPojoList(){
31         System.out.println("return json pojo list");
32         User user1=new User("aj",6);
33         User user2=new User("swift",52);
34         List<User> users=new ArrayList<>();
35         users.add(user1);
36         users.add(user2);
37         return users;
38     }
39 }

```

4.2 @ResponseBody 注解

- 名称: @ResponseBody
- 类型: **方法注解**
- 位置: SpringMVC控制器方法定义上方
- 作用: 设置当前控制器返回值作为响应体
- 范例:

```

@RequestMapping("/save")
@ResponseBody
public String save(){
    System.out.println("save...");
    return "{ 'info': 'springmvc' }";
}

```

四、REST风格

💡 Tip

REST(Representational State Transfer): 表征性状态转换。

RESTful: 使用REST风格对资源进行访问。

1.简介

- 传统风格 vs REST风格：
 - 传统风格的资源描述形式：
 - <http://localhost/user/getById?id=1>
 - <http://localhost/user/saveUser>
 - REST风格的资源描述形式：
 - <http://localhost/user/1>
 - <http://localhost/user>
- REST风格优点：
 - 隐藏资源的访问行为，无法通过url地址得知用户对资源执行何种操作。
 - 简化书写。
- 常见的REST风格的资源描述形式：

url地址	说明	行为动作（用于区分）
http://localhost/users	查询全部用户信息。	GET
http://localhost/users/1	查询指定用户信息。	GET
http://localhost/users	添加用户信息。	POST
http://localhost/users	修改用户信息。	PUT
http://localhost/users/1	删除指定用户信息。	DELETE

- **注意事项：**上述行为动作只是约定方式，而非规范，因此称作REST风格而不是REST规范。描述模块的名称通常使用复数，表示此类资源，而非单个资源，例如：users、books、accounts。

2.RESTful入门案例: @PathVariable

2.1 代码演示

```
1  @Controller
2  @RequestMapping("/users")// 请求路径前缀
3  public class UserController {
4
5      @RequestMapping(method = RequestMethod.POST)
6      @ResponseBody
7      public String save(@RequestBody User user) {
8          System.out.println("user save ... " + user);
9          return "{\"module':'user save'}";
10     }
11
12     // 我已设定了请求路径前缀
13     @RequestMapping(value =("/{id}", method = RequestMethod.DELETE)
14     @ResponseBody
15     public String delete(@PathVariable Integer id) {// @PathVariable表示
        示从url地址中取出id这个变量的值
16         System.out.println("user delete ... " + id);
17         return "{\"module':'user delete'}";
18     }
19
20     @RequestMapping(method = RequestMethod.PUT)
21     @ResponseBody
22     public String update(@RequestBody User user) {
23         System.out.println("user update ... " + user);
24         return "{\"module':'user update'}";
25     }
26
27     @RequestMapping(value =("/{id}", method = RequestMethod.GET)
28     @ResponseBody
29     public String getById(@PathVariable Integer id) {
30         System.out.println("user getById ... " + id);
31         return "{\"module':'user getById'}";
32     }
33 }
```

```

34     @RequestMapping(method = RequestMethod.GET)
35     @ResponseBody
36     public String getAll() {
37         System.out.println("user getAll ...");
38         return "{ 'module': 'user getAll' }";
39     }
40
41 }

```

2.2 注解区别

注解	作用	适用场景
<code>@RequestParam</code>	接收url地址传参或表单传参	发送非json格式的数据
<code>@RequestBody</code>	接收json传参	发送的请求参数超过1个且以json格式为主
<code>@PathVariable</code>	接收路径参数，使用 { 参数名称 } 来描述路径参数	采用RESTful进行开发，且参数数量较少。通常用于传递id值。

3.RESTful快速开发： `@RestController`、`@XxxMapping`

```

1  @RestController// 等价于@Controller + @ResponseBody
2  @RequestMapping("/books")
3  public class BookController {
4
5      //      @RequestMapping(method = RequestMethod.POST)
6      @PostMapping
7      public String save(@RequestBody Book book) {
8          System.out.println("book save ... " + book);
9          return "{ 'module': 'book save' }";
10     }
11

```

```

12 // @RequestMapping(value =("/{id}", method = RequestMethod.DELETE)
13 @DeleteMapping("/{id}")
14 public String delete(@PathVariable Integer id) { // @PathVariable表示
    从url地址中取出id这个变量的值
15     System.out.println("book delete ... " + id);
16     return "{ 'module': 'book delete' }";
17 }
18
19 @PutMapping
20 public String update(@RequestBody Book book) {
21     System.out.println("book update ... " + book);
22     return "{ 'module': 'book update' }";
23 }
24
25 @GetMapping("/{id}")
26 public String getById(@PathVariable Integer id) {
27     System.out.println("book getById ... " + id);
28     return "{ 'module': 'book getById' }";
29 }
30
31 @GetMapping
32 public String getAll() {
33     System.out.println("book getAll ...");
34     return "{ 'module': 'book getAll' }";
35 }
36 }

```

4.案例：基于RESTful的页面数据交互

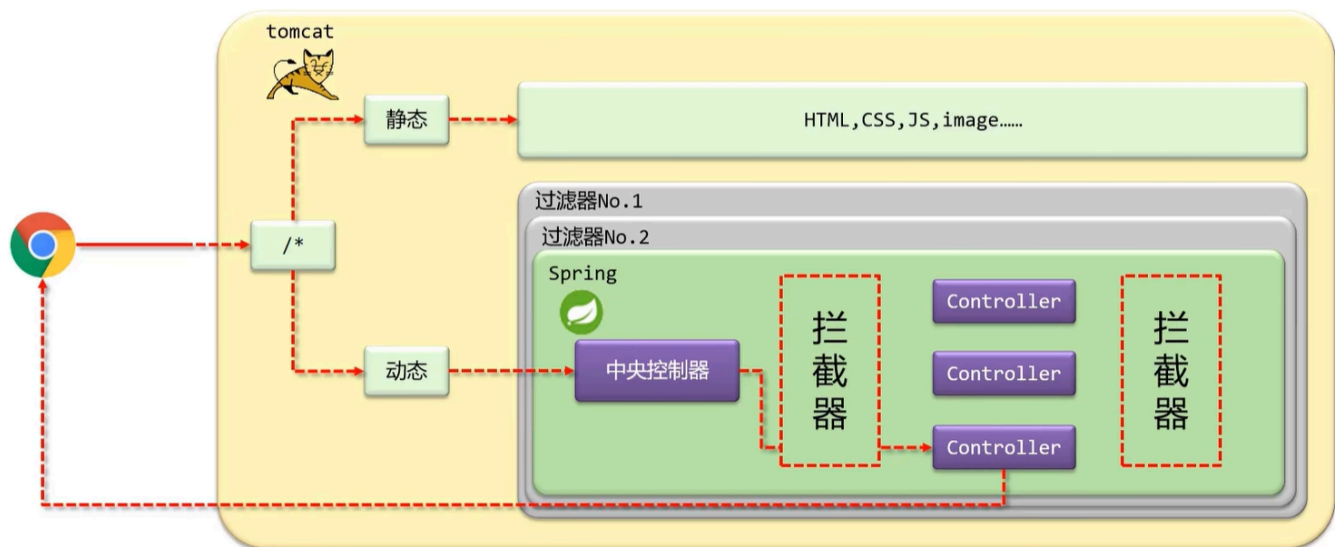
详见：[🔗springmvc\06_REST_case](#)

②：设置对静态资源的访问放行

```
@Configuration
public class SpringMvcSupport extends WebMvcConfigurationSupport {
    @Override
    protected void addResourceHandlers(ResourceHandlerRegistry registry) {
        // 当访问/pages/????时候, 走/pages目录下的内容
        registry.addResourceHandler("/pages/**").addResourceLocations("/pages/");
        registry.addResourceHandler("/js/**").addResourceLocations("/js/");
        registry.addResourceHandler("/css/**").addResourceLocations("/css/");
        registry.addResourceHandler("/plugins/**").addResourceLocations("/plugins/");
    }
}
```

五、拦截器

1. 概述



- **拦截器(Interceptor)**是一种动态拦截方法调用的机制，在SpringMVC中动态拦截控制器方法的执行。
- **作用：**
 - 在指定的方法调用前后执行预先设定的代码。
 - 根据需要阻止原始方法的执行。

- 拦截器与过滤器(Filter)的区别：
 - 归属不同：过滤器属于Servlet技术，拦截器属于SpringMVC技术。
 - 拦截内容不同：过滤器对所有访问进行增强，拦截器只针对SpringMVC的访问进行增强。
-

2.入门案例

2.1 创建拦截器类 `ProjectInterceptor`，并实现 `HandlerInterceptor` 接口

```
1  @Component
2  public class ProjectInterceptor implements HandlerInterceptor {
3      @Override
4      // 拦截之前
5      public boolean preHandle(HttpServletRequest request,
6      HttpServletResponse response, Object handler) throws Exception {
7          System.out.println("pre handle ...");
8          return true; // 若return false, 则会阻止原始方法的执行
9      }
10
11     @Override
12     // 拦截之后
13     public void postHandle(HttpServletRequest request,
14     HttpServletResponse response, Object handler, ModelAndView
15     modelAndView) throws Exception {
16         System.out.println("post handle ...");
17     }
18
19     @Override
20     // 完成后
21     public void afterCompletion(HttpServletRequest request,
22     HttpServletResponse response, Object handler, Exception ex) throws
23     Exception {
24         System.out.println("after completion ...");
25     }
26 }
```

2.2 创建配置类 `SpringMvcSupport`，继承 `WebMvcConfigurationSupport`，并实现 `addInterceptors()` 方法

```
1 | @Configuration
2 | public class SpringMvcSupport extends WebMvcConfigurationSupport {
3 |
4 |     @Autowired
5 |     private ProjectInterceptor interceptor;
6 |
7 |     @Override
8 |     protected void addInterceptors(InterceptorRegistry registry) {
9 |         // 访问/books及其子目录，都需要经过拦截器
10 |         registry.addInterceptor(interceptor).addPathPatterns("/books",
11 |             "/books/*");
12 |     }
13 | }
```

2.3 在 `SpringMvcConfig` 中扫描 `com.zsh.config` 这个包使拦截器生效

```
1 | @Configuration
2 | @ComponentScan({"com.zsh.controller", "com.zsh.config"})
3 | @EnableWebMvc
4 | public class SpringMvcConfig {
5 | }
```

2.4 使用标准接口 `WebMvcConfigurer` 简化2.2+2.3

⚠ Caution

此方法侵入式较强，与Spring紧耦合。

```
1 | @Configuration
```

```

2 // @ComponentScan({"com.zsh.controller", "com.zsh.config"})
3 @ComponentScan({"com.zsh.controller"})
4 @EnableWebMvc
5 public class SpringMvcConfig implements WebMvcConfigurer {
6     @Autowired
7     private ProjectInterceptor interceptor;
8
9     @Override
10    public void addInterceptors(InterceptorRegistry registry) {
11        // 访问/books及其子目录，都需要经过拦截器
12        registry.addInterceptor(interceptor).addPathPatterns("/books",
13            "/books/*");
14    }
15 }

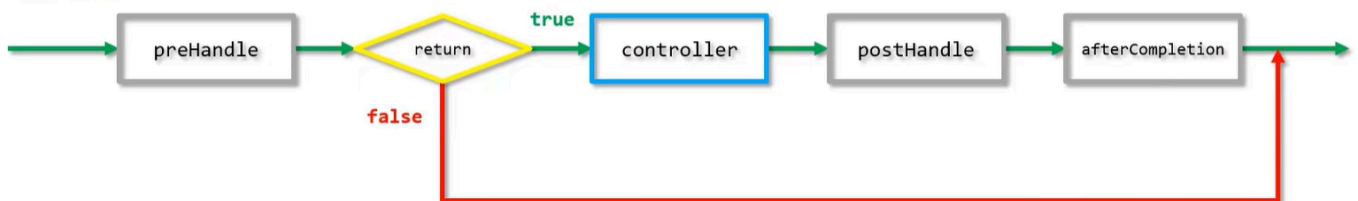
```

2.5 拦截器执行流程

无拦截器



有拦截器



3. 拦截器参数

3.1 前置处理器

```
1 public boolean preHandle(  
2     HttpServletRequest request,  
3     HttpServletResponse response,  
4     Object handler) throws Exception  
5 {  
6     System.out.println("pre handle ...");  
7     return true;  
8 }
```

- 参数：
 - `request`：请求对象。
 - `response`：响应对象。
 - `handler`：被调用的处理器对象，本质上是一个方法对象，对反射技术中的 `Method` 对象进行了再包装。
- 返回值：若 `return false`，则会阻止原始方法的执行。

3.2 后置处理器

```
1 public void postHandle(  
2     HttpServletRequest request,  
3     HttpServletResponse response,  
4     Object handler,  
5     ModelAndView modelAndView) throws Exception  
6 {  
7     System.out.println("post handle ...");  
8 }
```

- 参数 `modelAndView`：如果处理器执行完成后具有返回结果，则可以读取到对应的数据与页面信息，并进行调整（现代开发已淘汰）。

3.3 完成后处理器

```
1 public void afterCompletion(  
2     HttpServletRequest request,  
3     HttpServletResponse response,  
4     Object handler,  
5     Exception ex) throws Exception  
6 {  
7     System.out.println("after completion ...");  
8 }
```

- 参数 `ex`：如果处理器执行过程中出现异常对象，可以针对异常情况进行单独处理（现代开发已淘汰）。

4. 拦截器链的执行顺序

💡 Tip

当配置多个拦截器时，会形成拦截器链。

- 参照代码中拦截器的添加顺序，先进后出，后进先出（类似于“栈”）。
- 当某个拦截器中的 `preHandler()` 方法返回了 `true`，则**后面**拦截器均被终止运行，仅运行**前面**拦截器（不包括当前拦截器）的 `afterCompletion()` 方法。

多拦截器执行顺序

- 当配置多个拦截器时，形成拦截器链
- 拦截器链的运行顺序参照拦截器添加顺序为准
- 当拦截器中出现对原始处理器的拦截，后面的拦截器均终止运行
- 当拦截器运行中断，仅运行配置在前面的拦截器的afterCompletion操作

■ 按照1、2、3的顺序配置

- 全部返回成功
- 3返回false
- 2返回false
- 1返回false

